Quantorenlogik und Orthogonalität

Dieses Dokument führt die Konzepte der Datensatzqualität und der Quantorenlogik innerhalb des Example-Based-Testing (EBT) zusammen. Es dient als ultimative Referenz, um vergangene Fehlinterpretationen – insbesondere die hartnäckige Illusion, dass der Einsatz von Quantoren das Problem der Unterspezifikation automatisch löst – endgültig auszuräumen.
Wir beleuchten die Natur der beschränkten Existenz- und Allquantoren formal, übersetzen sie in die konkrete Architektur des QA-Bench-Frameworks und beweisen, dass die wahre Intelligenz der Qualitätssicherung nicht in der einzelnen Abfrage, sondern in der orthogonalen Konstruktion des gesamten Lösungsraums (Sigma-Stern Σ^*) liegt.

2.1 Formale Grundlagen der beschränkten Quantoren

Im klassischen Hoare-Tripel ${P}S{Q}$ repräsentieren P (Pre-Condition) und Q (Post-Condition) isolierte, eingefrorene Zustände. Sobald wir diese diskreten Punkte zu Mengen (Sets) erweitern, greifen wir auf die Prädikatenlogik erster Stufe (FOL) zurück, um den Raum der gültigen Transformationen abzustecken.

Der beschränkte Allquantor ($\forall$) – Das $N:1$ Mapping

Der Allquantor erzwingt Konvergenz. Er definiert, dass eine gesamte Menge an unterschiedlichen Eingangs-Zuständen (P_{set}) durch das Programmsegment (S) in exakt denselben, genormten Ziel-Zustand überführt werden muss.
$$\forall p \in P_{\text{set}} : S(p) = Q$$
Logische Operation: Eine deterministische **UND-Verknüpfung** (Konjunktion) aller aus den Transformationen resultierenden Assertions.

Der beschränkte Existenzquantor ($\exists$) – Das $1:N$ Mapping

Der Existenzquantor öffnet einen validen Lösungsraum. Er definiert, dass ein spezifischer Input auf eine vorab definierte Menge von erlaubten Zielzuständen (Q_{set}) abgebildet werden darf. Die Transformation ist erfolgreich, wenn der tatsächliche Output mit mindestens einem Element der Zielmenge identisch ist.
$$\exists q \in Q_{\text{set}} : S(P) = q$$
Logische Operation: Eine deterministische **ODER-Verknüpfung** (Disjunktion) der Assertions.

2.2 Quantoren in der EBT-Klasse (Praxis & Konvention)

In QA-Bench werden diese logischen Formeln nicht durch komplexe Metasprachen, sondern pragmatisch durch Python-Listen (`list`) umgesetzt. Das Framework iteriert über diese Listen, um die entsprechende Quantorenlogik anzuwenden.

Mapping	Parameter in EBT(...)	Wirkung der Engine
$N:1$ ($\forall$)	pre=[WorldState(...), ...]	Jeder pre-Zustand wird isoliert ausgeführt; alle müssen denselben post-Zustand ergeben (UND).
$1:N$ ($\exists$)	post=[WorldState(...), ...]	Der Output des pre-Zustands muss nur einem der gelisteten post-Zustände entsprechen (ODER).

Wichtige Architektur-Regel: Typ-Sicherheit in der ConventionMap

Die Standard-ConventionMap wertet eine Liste **nur dann** als formallogischen Quantor aus, wenn die Elemente innerhalb der Liste vom Typ WorldState sind (bzw. implizit zweifelsfrei dahin gecastet werden können). Eine Liste aus reinen Strings (z.B. pre=["datei1.txt", "datei2.txt"]) könnte je nach Konfiguration als schlichtes Listen-Argument für eine einzige Funktionsausführung interpretiert werden. Die explizite Deklaration als [WorldState(...), WorldState(...)] zwingt die Engine unmissverständlich in die iterative Sandbox-Orchestrierung der Quantoren.

2.3 Weitere Praxis-Beispiele

Um das Spektrum der Quantoren zu verdeutlichen, betrachten wir zwei Szenarien abseits der üblichen Datumsnormalisierung:

1. **Polymorphe Berechtigungssysteme ($N:1$ Mapping):** Ein System, das verschiedene Legacy-Nutzerrollen in einen standardisierten JWT-Token übersetzt.

```
EBT(
    target="auth:generate_token",
    pre=[
        WorldState(args=({"role": "super_admin"},)),
        WorldState(args=({"role": "legacy_root"},))
    ],
    post=WorldState(returns={"access_level": 0})
)
```

2. **Fallback-Parsing ($1:N$ Mapping):** Ein unzuverlässiger Third-Party-Endpoint liefert JSON, das manchmal genestet und manchmal flach ist. Das Parsing-Segment muss entweder das flache Modell oder das normalisierte genestete Modell zurückgeben, beide sind im weiteren Verlauf valide.

```
EBT(
    target="parser:parse_payload",
    pre=WorldState(fs={"/arena/input.json": "/host/raw_data.json"}),
    post=[
        WorldState(returns=FlatModel),
        WorldState(returns=NestedModel)
    ]
)
```

3.1 Die Vogelperspektive: Warum Quantoren die Unterspezifikation nicht lösen

Hier liegt der Kern des häufigsten Missverständnisses: **Ein einzelner Test-Fall mit Quantoren deckt zwar einen größeren Wertebereich ab, bleibt architektonisch aber dennoch nur eine einzige, isolierte EBT-Instanz.**
Aus der Vogelperspektive ist eine Definition wie "Alle diese 5 Chaos-Inputs ergeben den Output A" immer noch nur *eine* Koordinaten-Vorgabe in der Bibliothek von Babel. Das System erhält dadurch keine Informationen darüber, wie es sich verhalten soll, wenn ein Input auftritt, der *nicht* zu Output A führen soll. Die Schnittmenge möglicher Funktionen ist weiterhin riesig, und ein triviales (aber fehlerhaftes) Skript, das schlichtweg immer "Output A" zurückgibt (Overfitting), besteht den Test mühelos. Unterspezifikation ist ein Filter-Problem, und eine einzelne EBT-Instanz (selbst mit Quantor) ist ein zu grobmaschiges Netz.

3.2 Den Lösungsraum orthogonal aufspannen

Um das Problem der Unterspezifikation zu lösen, bedarf es keiner komplexeren Klassen im Framework, sondern schlichtweg **mehrerer orthogonaler EBT-Instanzen** innerhalb der Testsuite. Falsche Annahmen der Maschine kollabieren, wenn sie widersprüchlichen, inkompatiblen Szenarien ausgesetzt werden.

Beispiel A: Datums-Normalisierung

Eine einzige $N:1$ Instanz, die "12.04.2024" und "April 12th, 24" auf "2024-04-12" mappt, kann durch return "2024-04-12" ausgetrickst werden. Die Orthogonalität erfordert weitere Instanzen:

- **Instanz 1 (Der Happy Path $\forall$):** Mappt valide Datumsformate für den 12.04.2024 auf ISO.
- **Instanz 2 (Orthogonaler Shift $\forall$):** Mappt komplett andere Daten (z.B. den 31.12.1999) auf deren ISO-Äquivalent. (Zerstört den naiven return-Trick).
- **Instanz 3 (Der Randfall / Strukturbrecher):** Ein Schaltjahr-Datum (29.02.2024) wird korrekt verarbeitet.
- **Instanz 4 (Negative Koordinaten):** Ein offensichtlich ungültiges Datum ("32.13.2024") erzwingt eine Exception oder ein leeres Objekt.

Beispiel B: Der Zufallsgenerator

Ein $1:N$ Mapping erlaubt es einem 6-seitigen Würfel (Input) ein Ergebnis von 1 bis 6 (Output-Menge) zu generieren. Das Skript return 4 besteht diesen Test immer. Wir spannen den Raum auf:

- **Instanz 1 (Klassisch $\exists$):** W6 liefert 1, 2, 3, 4, 5 oder 6.
- **Instanz 2 (Orthogonale Parameter):** Wir ändern die Pre-Condition (einen W20 oder setzen definierte Min/Max-Bounds) und validieren gegen eine neue Q_{set}. (Zerstört den return 4-Trick für den W20).
- **Instanz 3 (Seed Injection):** Ein EBT-Test mit einem fest deklarierten RNG-Seed in pre, der zwingend exakt einen spezifischen post-Wert (1:1 Mapping) ergeben muss.

Disclaimer zum Zufallsgenerator: *Ein reines EBT (selbst mit orthogonalen Seeds und Bounds) kann lediglich beweisen, dass die Funktion technisch Zahlen innerhalb eines Rasters generiert. Es beweist nicht, dass die Verteilung der Zahlen tatsächlich stochastisch valide ist (z.B. ob der Würfel "gezinkt" ist). Ein defekter Generator, der nur 3, 4 und 5 abwechselnd auswirft, würde ein $1:N$ EBT-Mapping blind passieren, da die Ergebnisse Elemente der Zielmenge sind. Um echte Zufälligkeit zu testen, müsste die gesamte Testsuite des Generators in einen* ***stochastischen Wrapper*** *gepackt werden, der das Gesetz der großen Zahlen über n-Iterationen auswertet. Die genaue architektonische Implementierung und Case-Study hierfür (vermutlich über Derivate aus dem PBT) befindet sich aktuell noch in der Ausarbeitung.*

4.1 Zusammenfassung

Das Framework QABench ist keine magische Blackbox und bewahrt nicht vor den logischen Konsequenzen der Mengenlehre. Quantoren innerhalb einer EBT-Deklaration ($\forall$ über pre-Listen, $\exists$ über post-Listen) dienen lediglich dazu, deterministische Zustandsräume und Konvergenzen sauberer zu definieren, ohne Code zu duplizieren. Sie eliminieren jedoch nicht die Notwendigkeit, das System durch strategisch platzierte, radikal voneinander unabhängige Datenpunkte (Orthogonalität) in die Zange zu nehmen.

4.2 Die Testsuite als übergeordneter Allquantor

Formal betrachtet wird das Problem der Unterspezifikation gelöst, indem die Testsuite selbst als massiver, übergeordneter Allquantor auftritt, der über die Menge aller definierten EBT-Instanzen (T) iteriert. Während eine einzelne Instanz (t) intern $1:1$, $N:1$ oder $1:N$ Mappings abwickelt, verlangt die Architektur der Suite:
$$\forall t \in T : \text{eval}(t) = \text{True}$$
Je größer und orthogonaler die Menge T der Testfälle ist, desto enger wird der Korridor für das zu validierende Programmsegment S, bis nur noch die intendierte Funktion als einzige widerspruchsfreie Wahrheit aus der Schnittmenge übrig bleibt.

4.3 Konklusion

Die Maschine liefert am Ende aus der unendlichen Bibliothek von Babel immer nur exakt das, was die Orthogonalität deiner Datensätze kompromisslos erzwingt. Du bist der Architekt deiner Realität.